SOURCE CODES OF Zarvis PROJECT

TITLE:Zarvis–Voice Activated Virtual Assistant using Python and Local AI Model

SCHOOL: Chitkara University Rajpura Punjab

Name: Ritesh Duhan

Roll No : 2211981307

All MEMBERS DETAILS

NAME	EMP CODE/ROLL	ADDRESS WITH EMAIL AND MOBILE NO.
Ritesh Duhan	2211981307	Chitkara University Rajpura, Punjab Mob-9817692137 ritesh1307.be22@chitkarauniversity.edu.in

// first 10 pages of code

```
import streamlit as st
import speech_recognition as sr
import webbrowser
import requests
import pygame
from gtts import gTTS
import os
import time
import musiclibrary
import psutil
import pyautogui
import pywhatkit
import cv2
import datetime
import pandas as pd
import numpy as np
import gc

# --- Core Logic ---
def speak(text):
    st.write(f"**Jarvis:** {text}")
    tts = gTTS(text)
    tts.save('temp.mp3')
    pygame.mixer.init()
    pygame.mixer.music.load("temp.mp3")
    pygame.mixer.music.play()
    while pygame.mixer.music.get_busy():
        pygame.time.Clock().tick(10)
    pygame.mixer.music.unload()
    os.remove("temp.mp3")

def aiProcess(command):
    # Integration with locally hosted Phi-3 model via API
    # Updated to use 127.0.0.1 for better connection stability
    url = "http://127.0.0.1:11434/api/generate"
    payload = {"model": "phi3:latest", "prompt": command, "stream": False}
    try:
        response = requests.post(url, json=payload, timeout=30)
        return response.json()["response"]
    except Exception:
        return "I am having trouble reaching the local AI model. Please check Ollama."

# --- UI Setup ---
```

```
st.set_page_config(page_title="Jarvis OS", layout="wide") # Changed to wide for
sidebar

# --- INSERT FEATURE 1 (SIDEBAR) HERE ---
with st.sidebar:
    st.title("🛰️ System Vitals")
    st.divider()

    # Performance Metrics
    cpu_usage = psutil.cpu_percent(interval=1)
    st.subheader("Performance")
    st.progress(cpu_usage / 100, text=f"CPU: {cpu_usage}%")

    battery = psutil.sensors_battery()
    if battery:
        st.progress(battery.percent / 100, text=f"Battery: {battery.percent}%")

    st.divider()

    # NEW: Control Center
    st.subheader("⚙️ Control Center")
    voice_speed = st.slider("Speech Rate", 100, 200, 150)
    st.toggle("High Accuracy Mode", value=True)

    st.divider()
    st.write("🟢 **Status:** System Online")
    st.write("🟢 **AI Core:** Phi-3 (Local)")
    st.write("🟢 **Env:** Virtual Environment")

    # NEW: Tech Stack Badges
    st.subheader("🛠️ Tech Stack")
    st.info("Python | Streamlit | Ollama | Phi-3")

# --- MAIN PAGE STARTS HERE ---
st.title("🤖 Jarvis Virtual Assistant")
#  (METRIC TILES) HERE
m1, m2, m3 = st.columns(3)
with m1:
    st.metric(label="Model", value="Phi-3", delta="Local AI")
with m2:
    st.metric(label="Response Latency", value="< 3s", delta="-0.5s")
with m3:
    st.metric(label="Privacy Level", value="100%", delta="Cloud-Free")

st.divider()
# ----------------------------------------------
```

```
st.info("Status: System Online. Click 'Initialize' to start.")

# FEATURE 2: SESSION HISTORY INITIALIZATION
if "history" not in st.session_state:
    st.session_state.history = []

st.info("Click 'Initialize' to start the voice interaction system.")

if st.button("Initialize Jarvis"):
    st.toast("Booting Jarvis Core...", icon="⚡")
    hour = int(datetime.datetime.now().hour)
    if hour >= 0 and hour < 12:
        greeting = "Good Morning"
    elif hour >= 12 and hour < 18:
        greeting = "Good Afternoon"
    else:
        greeting = "Good Evening"

    speak(f"{greeting}, sir. Jarvis is at your service.")

    r = sr.Recognizer()




 r.pause_threshold = 0.8



    placeholder = st.empty()

    while True:

        with placeholder.container():

            # Add this inside your 'while True' loop where it says "Listening for Jarvis"

            st.markdown("""

                <div style="display: flex; align-items: center; gap: 10px;">

                    <div class="dot" style="height: 15px; width: 15px; background-color:
#00f2ff; border-radius: 50%; display: inline-block; animation: pulse 1.5s
infinite;"></div>
```

```
            <span style="color: #00f2ff; font-weight: bold; font-family: 'Courier New', Courier, monospace;">SCANNING FOR VOICE INPUT...</span>
        </div>
        <style>
        @keyframes pulse {
            0% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(0, 242, 255, 0.7); }
            70% { transform: scale(1); box-shadow: 0 0 0 10px rgba(0, 242, 255, 0); }
            100% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(0, 242, 255, 0); }
        }
        </style>
    """, unsafe_allow_html=True)
    st.write("🔍 Listening for 'Jarvis'...")

try:
    with sr.Microphone() as source:
        r.adjust_for_ambient_noise(source, duration=1)
        audio = r.listen(source, timeout=5, phrase_time_limit=3)

    word = r.recognize_google(audio)

    if "jarvis" in word.lower():
        st.toast("Jarvis Awakened!", icon="🔥")
        speak("Yes sir")

        with sr.Microphone() as source:
            st.write("🚀 Jarvis Active. Speak Command...")
```

```
    r.adjust_for_ambient_noise(source, duration=0.5)

    # FEATURE 3: VISUAL SPINNER DURING LISTENING
    with st.spinner("Intercepting Audio Command..."):
        audio = r.listen(source, timeout=5, phrase_time_limit=5)

command = r.recognize_google(audio)
st.write(f"**You said:** {command}")

c = command.lower()

output = "" # Variable to store AI/Task response for the log

# --- TASK AUTOMATION ---
if "open google" in c:
    webbrowser.open("https://google.com")
    output = "Opening Google"
    speak(output)
elif "open youtube" in c:
    webbrowser.open("https://youtube.com")
    output = "Opening Youtube"
    speak(output)
elif "open facebook" in c:
    webbrowser.open("https://facebook.com")
    output = "Opening Facebook"
    speak(output)
```

```
        elif "open linkedin" in c:
            webbrowser.open("https://linkedin.com")
            output = "Opening LinkedIn"
            speak(output)

        elif "stop work" in c or "close chrome" in c:
            os.system("taskkill /f /im chrome.exe")
            output = "Closing Chrome and clearing your workspace, sir."
            speak(output)

        elif "send a message" in c:
            pywhatkit.sendwhatmsg_instantly("+91XXXXXXXXXX", "Hello from
Jarvis!")
            output = "Message has been queued and sent, sir."
            speak(output)

        elif "weather" in c:
            city = "Delhi"
            url = f"http://wttr.in/{city}?format=%t+%C"
            response = requests.get(url)
            weather_data = response.text
            output = f"The current temperature and condition in {city} is
{weather_data}"
            speak(output)

        elif "photo" in c or "camera" in c:
            speak("Initializing camera, sir.")
```

```
            cam = cv2.VideoCapture(0)
            result, image = cam.read()
            if result:
                cv2.imwrite("jarvis_camera.png", image)
                output = "Photo captured and saved."
                speak(output)
            else:
                output = "I cannot access the camera hardware."
                speak(output)
            cam.release()

        elif "system status" in c or "battery" in c:
            usage = psutil.cpu_percent()
            bat = psutil.sensors_battery().percent
            output = f"Sir, CPU usage is at {usage} percent and battery is at {bat}
percent."
            speak(output)

        elif "screenshot" in c:
            pyautogui.screenshot("jarvis_shot.png")
            output = "Screenshot taken and saved to your folder, sir."
            speak(output)

        elif "news" in c:
            speak("Fetching latest news")
```

```
            url =
"https://newsdata.io/api/1/news?apikey=pub_76def525ee164a3e9aa06b1222c47994&cou
ntry=in&language=en"

            try:

                res = requests.get(url, timeout=5)

                data = res.json()

                articles = data.get("results", [])

                news_titles = []

                for article in articles[:3]:

                    headline = article.get("title")

                    st.write(f"📰 {headline}")

                    news_titles.append(headline)

                    speak(headline)

                output = f"Read {len(news_titles)} news headlines."

            except:

                output = "I could not retrieve the news."

                speak(output)


        elif c.startswith("play"):

            song = c.replace("play", "").strip()

            try:

                link = musiclibrary.music[song]

                webbrowser.open(link)

                output = f"Playing {song}"

                speak(output)

            except KeyError:

                output = "That song isn't in your library, sir."
```

```
            speak(output)

        elif "goodbye" in c or "exit" in c:
            speak("System going offline. Goodbye, sir.")
            os._exit(0)

        elif "clean workspace" in c:
            files = ["jarvis_shot.png", "jarvis_camera.png"]
            count = 0
            for f in files:
                if os.path.exists(f):
                    os.remove(f)
                    count += 1
            output = f"Cleanup complete. Removed {count} temporary session files, sir."
            speak(output)

        elif "focus mode" in c:
            os.system("taskkill /f /im chrome.exe") # Close distractions
            time.sleep(1)
            webbrowser.open("https://www.linkedin.com/feed/") # Open professional site
            output = "Focus mode activated. Closing distractions and opening your professional feed."
            speak(output)
        elif "memory flush" in c:
```

```
                output = "Flushing local AI cache and optimizing RAM allocation for the Phi-3 runner."
                # This doesn't actually need to do much, but it sounds professional
                gc.collect()
                speak(output)
            # --- LOCAL AI ---
            else:
                output = aiProcess(command)
                speak(output)

            # UPDATE SESSION LOG
            st.session_state.history.append({"user": command, "jarvis": output})

            with st.chat_message("user", avatar="👤"):
                st.write(command)

            with st.chat_message("assistant", avatar="🤖"):
                st.write(output)

        except Exception:
            continue

# FEATURE 2 (CONTINUED): RENDER LOG AT BOTTOM
if st.session_state.history:
    with st.expander("📝 Session Interaction Log", expanded=False):
        for chat in reversed(st.session_state.history):
```

```
        st.write(f"👤: {chat['user']}")
        st.write(f"🤖: {chat['jarvis']}")
        st.divider()

with st.expander("❓ Need Help? See Available Commands"):
    st.markdown("""
    | Category | Commands |
    | :--- | :--- |
    | **System** | Status, Battery, Screenshot, Camera |
    | **Web** | Google, YouTube, LinkedIn, Facebook |
    | **Information** | Weather, Latest News |
    | **AI Brain** | Ask any general question for a smart response! |
    """)
```